



Paradigmas de Programación

Recursividad y Orden Superior

Docente: Pennella Valeria

Inducción/Recursión

- ❖ El concepto de **recursividad** viene de la mano de inducción: Verifico para $P(0)$ y dado un $P(N)$ se puede verificar para $P(N+1)$.
- ❖ La **inducción** es un mecanismo que nos permite:
 - ❖ Definir conjuntos infinitos
 - ❖ Definir funciones recursivas sobre ellos, con garantía de terminación
 - ❖ Probar propiedades sobre sus elementos

- Que una función este definida recursivamente, significa que dicha función se auto invoca.
- Ejemplo clásico: **Factorial**

$$n! = 1 \times 2 \times 3 \times \dots \times (n - 1) \times n$$

Sabemos que:

$$0! \text{ es } 1$$

$$n > 0 \text{ es } n * (n - 1)!$$

Implementado en Haskell

`factorial 0 = 1`  Caso Base

`factorial n = n * factorial (n - 1)`



Caso Inductivo

Recursividad con listas

Conocer la cantidad de elementos de la lista

`length [] = 0`

`length (x:xs) = 1 + length xs`

- Conocer el último elemento de la lista:

```
last [x] = x
```

```
last (x:xs) = last xs
```

- Saber si un elemento pertenece a un conjunto:

```
elem _ [] = False
```

```
elem e (x:xs) = e == x || elem e xs
```

- Dar vuelta los elementos de la lista

```
reverse [] = []
```

```
reverse (x:xs) = (reverse xs) ++ [x]
```

```
Main> reverse ["maria", "juan", "ana"]  
["ana", "juan", "maria"]
```

- **Conocer el máximo de una lista:**

```
maximo [x] = x
```

```
maximo (x:xs) = x `max` maximo xs
```

```
Main> maximo [4,5,7,2,1]
```

```
7
```

Funciones de Orden Superior definidas con recursividad ej:

- Transformar una lista en otra aplicando una función (map)

```
map :: (a -> b) -> [a] -> [b]
```

```
map [] = []
```

```
map f (x:xs) = f x : map f xs
```

- Seleccionar los elementos que cumplen con una condición (filter)

```
filter :: (a -> Bool) -> [a] -> [a]
```

```
filter [] = []
```

```
filter f (x:xs) | f x = x : filter f xs  
                | otherwise = filter f xs
```

- **Verificar si todos los elementos cumplen con una condición.**

```
all :: (a -> Bool) -> [a] -> Bool
```

```
all _ [] = True
```

```
all f (x:xs) = f x && all f xs
```

- **Verificar si alguno de los elementos cumple con una condición.**

```
any :: (a -> Bool) -> [a] -> Bool
```

```
any _ [] = False
```

```
any f (x:xs) = f x || any f xs
```

Resumen

- Inducción.
- Recursividad sin listas, ej factorial.
- Aprovechamiento del Pattern Matching para descomponer tipos compuestos como listas.
- Recursividad con listas y para definir funciones sin orden superior como la función máximo.
- Recursividad con listas y para definir funciones de orden superior como any, all.